PROJECT TITLE	**:** HTML QUICK STYLER
TEAM ID	**:** 69a929b0fe45d7d8dad41633
TEAM MEMBERS	**:** DIVYA DHARSHINI C AKSHAYA VARSHINI M B KIRUTHIKA G RENUGA DEVI M PRIYA DHARSHINI P
COLLEGE NAME	**:** S. A. V SAHAYA THAI ARTS AND SCIENCE (WOMEN) COLLEGE
COLLEGE CODE	**:** 138
DATE	**:** 09.03.2026

HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

PROJECT DESCRIPTION:

The **Smart Web Layout Generator** is a web-based application developed using **Python and the Gradio framework** that helps users create modern website layouts automatically. The main goal of this project is to simplify the process of designing HTML webpages for beginners, students, and developers who may not have advanced web development knowledge.

In traditional web development, creating a website requires writing a large amount of **HTML and CSS code manually**. This can be difficult and time-consuming, especially for beginners. The Smart Web Layout Generator solves this problem by allowing users to simply enter basic information such as the **website title and website description**, and the system automatically generates a complete **HTML webpage structure**.

The generated webpage includes several important sections such as a **header section, about section, services section, contact section, and footer**. These sections follow common website design practices and help create a professional-looking webpage.

The system also includes additional features such as **live preview functionality**, where users can instantly see the generated webpage inside the application interface. This helps users understand how the website will appear in a real browser.

SCENARIOS

Scenario 1: Portfolio Website Generation

A student or freelancer may want to create a **personal portfolio website** to showcase their skills, projects, and achievements. Designing a website manually requires knowledge of HTML and CSS, which can be difficult for beginners.

Using the **Smart Web Layout Generator**, the user simply enters the **website title and description** in the Generate Page section. After clicking the **Generate Page** button, the system automatically creates a structured HTML webpage.

Scenario 2: Startup Landing Page

A startup company may need a simple **landing page** to introduce their product or service to potential customers.

Using the Smart Web Layout Generator, the startup team enters the **product or company name as the website title** and writes a short **description explaining the product or service**.

After generating the webpage, the system automatically creates a **modern webpage layout** containing a header section, an about section describing the product, a services section that highlights key features, and a footer section.

Scenario 3: Educational Website

Teachers, trainers, or educational institutions may want to create a **simple webpage to present course information or learning resources**.

Using the Smart Web Layout Generator, the teacher enters the **course title and description** in the Generate Page tab. The system then automatically generates a structured webpage.

ARCHITECTURE OVERVIEW

The **Smart Web Layout Generator** system is designed using a **modular architecture** that combines Python programming, HTML generation, CSS styling, and a web-based user interface.

Python is used as the **core programming language** to implement the main application logic. It handles user inputs such as the website title and description, processes the data, and dynamically generates the HTML webpage structure.

The **Gradio framework** is used to build the interactive web interface. It provides a simple and user-friendly environment where users can enter website details, generate webpage layouts, and preview the generated HTML directly in the browser.

The system contains multiple functional modules that work together to generate a complete webpage. The **HTML generator module** creates a structured webpage template containing sections such as header, about, services, contact, and footer. The **preview module** allows users to instantly render the generated HTML code and view the webpage layout.

Core Technologies

• **Python 3.x:** Used for implementing the main application logic and HTML generation.

• **Gradio:** Used to create the interactive web interface for user input and preview.

• **HTML5:** Used to structure the generated web pages.

• **CSS3:** Used for styling the webpage layout and design.

• **Web Browser:** Used to display the generated webpage and live preview.

PRE-REQUISITES

Software Requirements

❖ **Python 3.8+:** Required to run the application.

❖ **Gradio Library:** Install using pip command

```
pip install gradio
```

❖ **Web Browser:** Chrome, Firefox, Edge, or Safari to run the interface.

❖ **Internet Connection:** Required for installing libraries and running the application.

Knowledge Prerequisites

Basic knowledge of the following concepts is useful:

• Python programming fundamentals

• HTML webpage structure

• CSS styling basics

• Basic understanding of Gradio interface

These concepts help users understand how the system generates HTML web pages automatically.

Hardware Requirements

Minimum system requirements include:

• **Processor:** Intel i3 or equivalent

• **RAM:** Minimum 4 GB

• **Storage:** At least 1 GB free space

• **Display Resolution:** 1280 × 720 or higher

PROJECT WORKFLOW

The development of the **Smart Web Layout Generator** follows a structured workflow that includes environment setup, application development, interface design, deployment, and testing. Each phase focuses on building important components of the system.

The workflow ensures that the system is developed in a **modular and organized manner**, making it easier to maintain, test, and extend in the future.

The main phases of the project include:

• Environment Setup
• Core Application Development
• Gradio Interface Development
• Deployment
• Testing and Validation

Each phase contributes to building a complete **web layout generation system**.

Phase 1: Environment Setup & Model Configuration

This phase focuses on preparing the development environment required for building and running the application.

The project can be developed using **Google Colab, Jupyter Notebook, or a local Python environment**. These environments allow developers to write and execute Python code easily.

During this phase, the required Python libraries are installed and configured.

Activity 1.1: Install Dependencies and Setup Environment

Before implementing the application, necessary Python packages must be installed. These libraries provide functionalities for building the web interface and running the application.

The main library required for this project is:

• Gradio

The following command is used to install the library:

```
pip install gradio
```

After installing the library, the development environment becomes ready for implementing the project.

Activity 1.2: Configure Gradio Application

In this step, the **Gradio framework** is configured to create the interactive user interface for the Smart Web Layout Generator.

Gradio allows developers to build web-based interfaces for Python applications easily. It connects the backend Python functions with frontend interface components such as **text boxes, buttons, tabs, and preview panels**.

In this project, the Gradio **Blocks layout system** is used to organize the interface into multiple tabs. Each tab represents a different functionality of the application.

Phase 2: Core Application Development

This phase focuses on building the main components of the system including **HTML page generation, CSS styling, and layout structure**.

These components work together to generate complete web pages based on user inputs.

Activity 2.1: Color Palette Generator

The system includes a **Color Palette Viewer** that allows users to explore different color themes for website design.

Each palette displays predefined color combinations that represent the **primary color, secondary color, and background color** used in webpage design. These palettes help users understand how different color schemes can be used to create visually attractive websites.

Some of the palettes included in the system are:

- Modern
- Ocean
- Sunset
- Forest

When the user clicks the **Show Color Palettes** button, the system displays multiple palette cards showing the color blocks and their corresponding color codes.

```
# -------- COLOR PALETTES --------

def show_palette():

    return """
<div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:20px">

<div style="padding:20px;border-radius:10px;background:white;box-shadow:0 4px 10px rgba(0,0,0,0.1)">
<h3>Modern</h3>
<div style="height:30px;background:#4facfe"></div>
<div style="height:30px;background:#00f2fe"></div>
<p>Primary : #4facfe</p>
<p>Secondary : #00f2fe</p>
<p>Background : #f5f5f5</p>
</div>

<div style="padding:20px;border-radius:10px;background:white;box-shadow:0 4px 10px rgba(0,0,0,0.1)">
<h3>Ocean</h3>
<div style="height:30px;background:#2193b0"></div>
<div style="height:30px;background:#6dd5ed"></div>
<p>Primary : #2193b0</p>
<p>Secondary : #6dd5ed</p>
<p>Background : #e0f7fa</p>
</div>
```

Activity 2.2: HTML Page Builder

The **HTML Page Generator** module is responsible for creating the basic HTML structure of the website.

This module dynamically inserts user-provided information such as the **website title and description** into a predefined HTML template using Python.

The generated webpage includes important sections such as:

• Header Section
• About Section
• Services Section
• Contact Section
• Footer Section

The **header section** displays the main title and description of the website. The **about section** provides basic information about the website, while the **services section** highlights key features or services. The **contact section** provides communication details, and the **footer section** displays copyright information.

```
with gr.Blocks() as demo:

    gr.Markdown("# Smart Web Layout Generator")

    with gr.Tabs():

        with gr.Tab("Generate Page"):

            title = gr.Textbox(label="Website Title")
            description = gr.Textbox(label="Website Description")

            generate_btn = gr.Button("Generate Page")

            html_code = gr.Textbox(lines=20,label="Generated HTML Code")

            generate_btn.click(
                generate_page,
                inputs=[title,description],
                outputs=html_code
            )


        with gr.Tab("Live Preview"):
```

Activity 2.3: CSS Styling Engine

To improve the visual appearance of the generated website, the system includes a **CSS Styling Module** within the HTML template.

The CSS rules define the **layout, fonts, colors, spacing, and design elements** used in the webpage. These styles help create a clean, modern, and visually attractive interface.

Some of the styling features include:

• Gradient header design
• Grid layout for service sections
• Card-style feature blocks with shadows
• Rounded corners and spacing for sections

By automatically applying CSS styles, the system ensures that the generated websites look **professional and well-structured** without requiring users to write CSS code manually.

```
# -------- HTML GENERATOR --------

def generate_page(title, description):

    html = f"""
<!DOCTYPE html>
<html>
<head>

<title>{title}</title>

<style>

body {{
font-family: Arial;
margin:0;
background:#f5f5f5;
}}

header {{
background:linear-gradient(90deg,#4facfe,#00f2fe);
color:white;
text-align:center;
padding:60px;
```

Phase 3: Gradio Interface Development

The **Smart Web Layout Generator** application uses **Gradio** to build an interactive user interface.

Gradio allows developers to create **web-based interfaces for Python applications** easily. Through this interface, users can interact with the system and generate website layouts without writing any HTML or CSS code.

The interface is designed using **multiple tabs** to organize different functionalities of the application. Each tab provides a specific feature of the system such as webpage generation, preview, color palettes, templates, and usage guide.

Tab 1: Generate Page

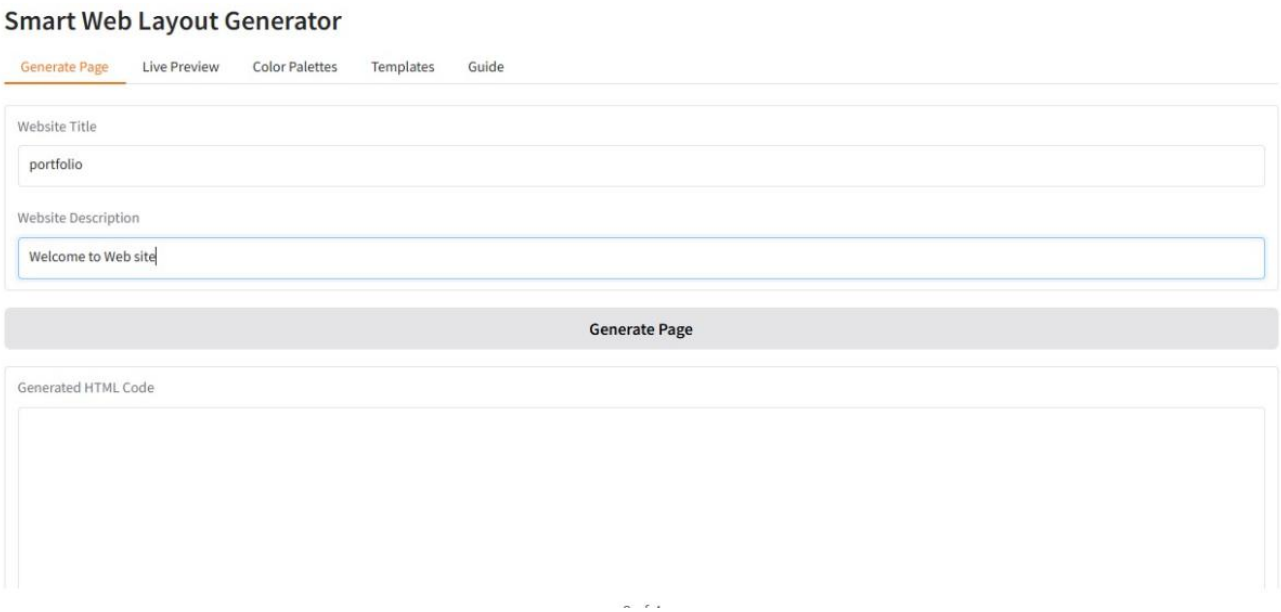
This tab allows users to generate a website layout automatically.

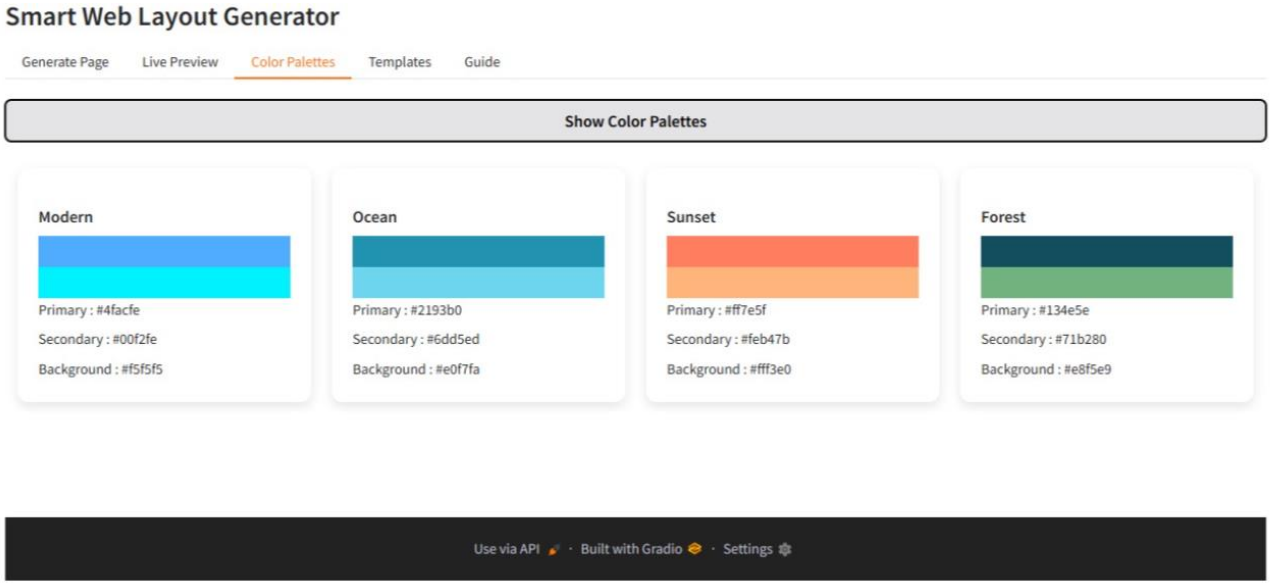
Users can enter:

• Website Title
• Website Description

Once the user clicks the **Generate Page** button, the system generates a **structured HTML webpage** based on the provided inputs.

The generated HTML code is displayed in the output section, allowing users to view or copy the code for further use.


Smart Web Layout Generator
Generate Page
Live Preview
Color Palettes
Templates
Guide
Website Title
portfolio
Website Description
Welcome to Web site
Generate Page
Generated HTML Code

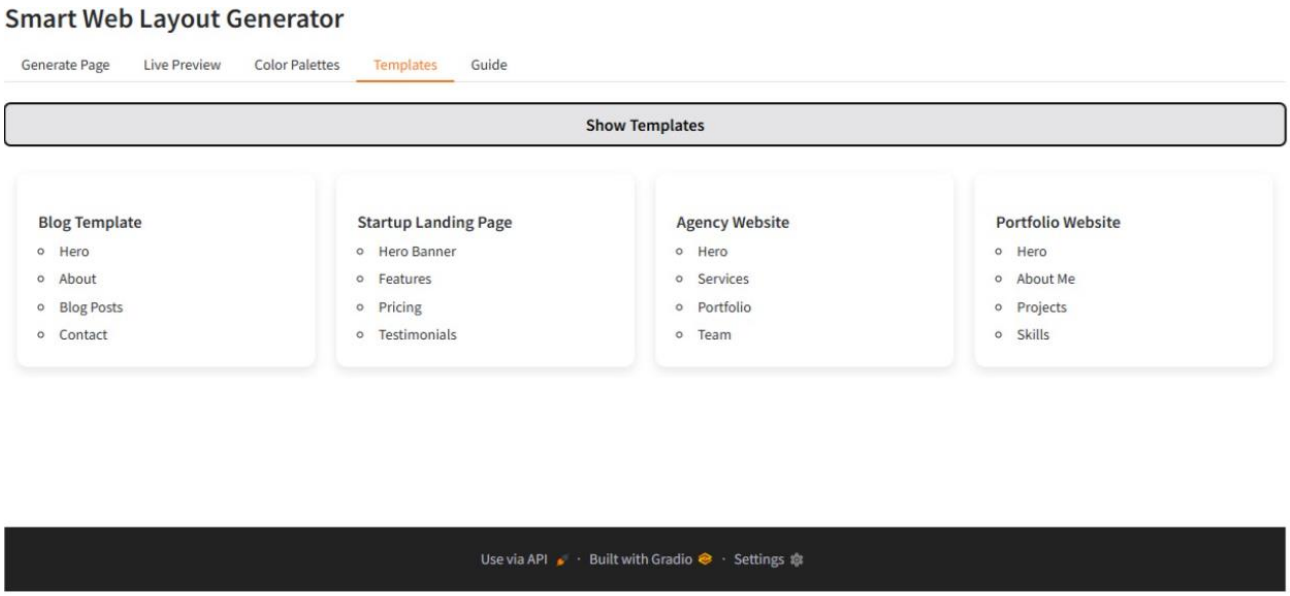

Smart Web Layout Generator
Generate Page
Live Preview
Color Palettes
Templates
Guide
Show Color Palettes
Modern
Primary : #4facfe
Secondary : #00f2fe
Background : #f5f5f5
Ocean
Primary : #2193b0
Secondary : #6dd5ed
Background : #e0f7fa
Sunset
Primary : #ff7e5f
Secondary : #feb47b
Background : #fff3e0
Forest
Primary : #134e5e
Secondary : #71b280
Background : #e8f5e9
Use via API · Built with Gradio · Settings

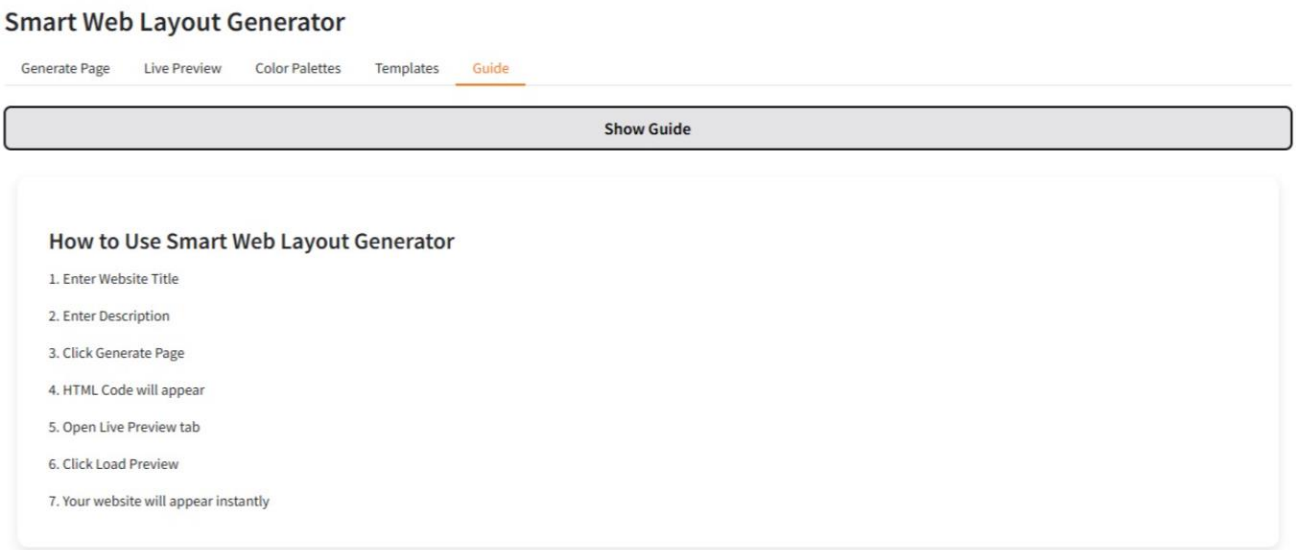
Tab 2: AI Generator

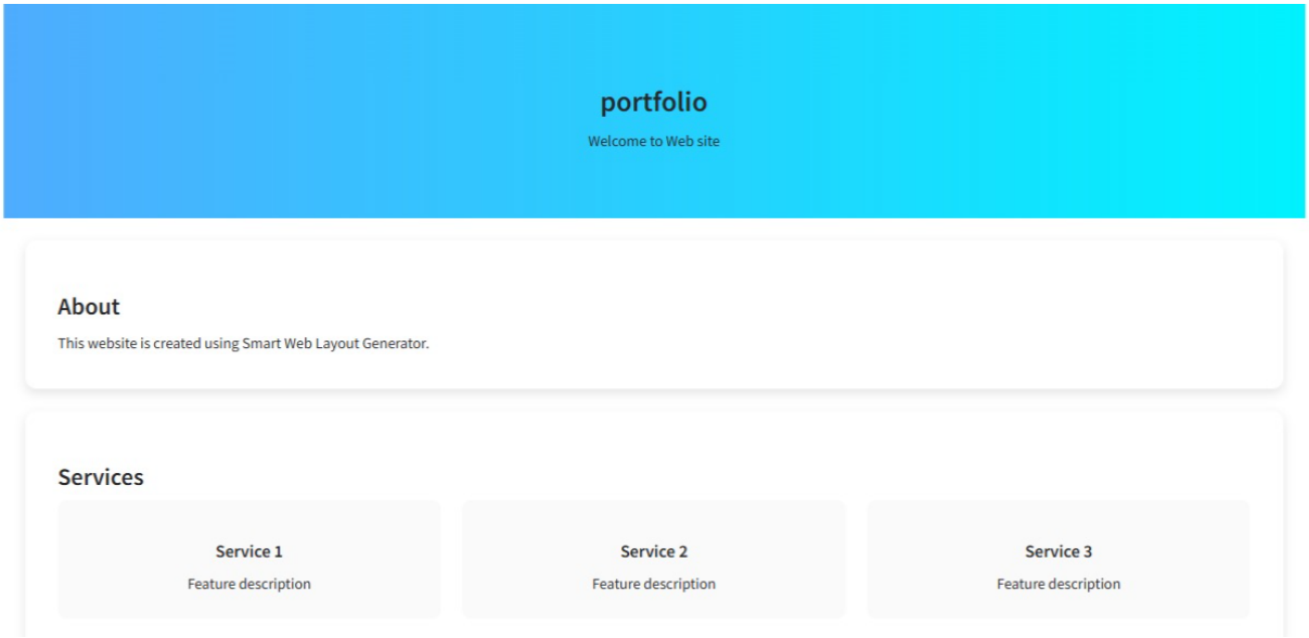
This tab uses the IBM Granite AI model to generate HTML code automatically.

Users provide a title and description for the website, and the system generates a complete HTML webpage using AI.

This feature demonstrates how artificial intelligence can assist in generating website layouts dynamically.







Tab 3: Template Guide

This tab provides information about different types of website layouts that can be created using the system.

It explains common website structures such as:

➢ **Blog Template** – includes sections for posts, about information, and contact details.

➢ **Startup Landing Page** – includes hero section, features, pricing, and testimonials.

➢ **Agency Website** – includes services, portfolio, and team sections.

➢ **Portfolio Website** – includes personal introduction, skills, and project showcase.

These templates help users understand different **website design structures** and give ideas for building their own webpages.

Phase 4: Deployment

After implementing all system components, the application can be deployed using **Python and Gradio**.

Gradio generates a **local or public web interface link** that allows users to access the application through a web browser.

The application can be executed in environments such as **Google Colab, Jupyter Notebook, or a local Python environment**. Once the program runs, Gradio launches the interface and displays a link that users can open in a browser.

This allows users to easily access the **Smart Web Layout Generator** without installing additional software.

Phase 5: Testing and Validation

Testing is performed to ensure that the system functions correctly and generates valid HTML webpages.

Functional testing verifies that:

• User inputs such as title and description are accepted correctly

• HTML webpages are generated successfully

• The live preview displays the webpage properly

• Color palettes and template information are displayed correctly

• The interface works smoothly across different browsers

Different input values are tested to ensure that the system produces the expected output and behaves correctly in different situations.

.

Conclusion

The **Smart Web Layout Generator** project demonstrates how Python and Gradio can simplify the process of creating website layouts.

The system allows users to generate structured HTML webpages quickly by simply entering basic information such as the website title and description. The generated webpage includes important sections like header, about, services, contact, and footer.

Additional features such as **live preview, color palette suggestions, template information, and user guide** help users design visually appealing websites easily.

Overall, this project provides a **simple, efficient, and beginner-friendly solution** for generating modern web page layouts without requiring advanced coding knowledge.

SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product